

产品需求文档

1. 背景介绍

根据前期围绕时间管理 App 开展的市场调研，团队对具有代表性的时间管理产品进行了体验和对比。按照产品解决问题的侧重点，可将这些产品归纳为五类：以待办清单为核心的任务管理类、以日历和时间块为核心的日程规划类、以番茄钟和时间追踪为核心的专注执行类、以目标拆解和习惯养成为核心的目标管理类，以及利用 AI 自动安排任务与时间的智能编排类。部分团队协作产品也提供任务和日程能力，但其主要服务于多人协同和项目推进。

进一步对比各类产品后可以发现：任务管理类擅长记录，却仍需要用户自行判断先后顺序；日程规划类能够安排时间，但计划发生变化后需要频繁手动维护；专注执行类帮助用户进入执行状态，却较少解决做什么和何时做的问题；目标管理类适合长期目标推进，但学习和维护成本较高；智能编排类能够自动排程，却往往功能复杂或使用成本较高。因此，市场仍缺少一种面向个人、学习成本较低，并能同时完成任务聚合、优先级决策和计划动态调整的时间管理方案。

结合 XEngineer 训练营学员任务密集、变化频繁、需要快速投入执行的实际场景，产品最终定位为一款面向训练营学员的个人时间管理 App。产品聚焦三个核心环节：汇集用户需要处理的任务，生成清晰且可解释的今日执行顺序，并在任务延期、阻塞或状态变化后自动调整计划，帮助用户持续明确下一步应该做什么，降低安排和维护计划的心智负担。

2. 产品目标

业务目标

- 跑通个人真实会用的最短路径。
- 验证“GitHub-only”模式在个人微观任务管理场景下的可行性。
- 核心指标：用户对推荐任务顺序的采纳率、任务自动重排建议的接受率、日活跃用户占比。

用户目标

- 让用户无需手动整理任务，避免遗漏，且确保任务状态及时与 GitHub 联动。
- 让用户在打开工具的第一眼，就能明确当前需要推进的 1-3 个最优先行动，并清晰理解排序理由。
- 在计划失效或被阻塞时，用户无需手动重新排期即可直接获得新的行动指南。

3. 目标用户

用户群体：XEngineer 训练营学员。

用户特征

- 拥有 Github 账号且严格遵循《Github 管理规范》
- 处于高强度、快节奏的学习与开发节奏中，需要及时更新任务及其任务状态
- 需要快速梳理任务以及完成顺序。

4. 用户痛点/需求分析

用户现在有什么问题？

- 跨项目整理任务繁琐：**参与多个项目时，用户需要去查看多个项目的 Issue，并手动复制到其他地方记录，严重降低了工作效率。
- 面对任务列表无从下手：**将 GitHub 的 Issue 和 PR 导入后，面对长长的任务列表，用户依然不知道应该先做哪一个。
- 计划频繁被打断与变更：**实际开发执行经常与早上的计划不符，例如任务未按时完成、被阻塞、新任务进入、原有任务硬截止发生变化，或者提交的 PR 收到 `changes requested`。
- 心智负担过重：**所有的优先级判定和计划调整都依赖用户大脑记忆，一旦发生新变化（如 DDL 修改、被人 Request

Review)，脑子里的优先级就会马上乱掉。

用户为什么需要这些功能？

- **实现任务自动聚合（自动导入）**：用户需要系统单向自动同步分配给自己的 GitHub Issue，省去手动记录的时间，并防止任务遗漏。
- **获取可信赖的执行指引（优先级排序）**：用户需要一个能明确告诉自己“先做什么”的列表，并且这个排序必须提供能看懂、能核对的解释依据，而不是黑盒盲猜。
- **降低计划维护成本（任务自动重排）**：当任务发生变化后，用户希望无需重新排期和确认，就能立即看到调整后的任务列表，并清楚知道任务变动的原因，从而直接继续执行。

这些问题出现在哪些场景？

- **项目初期或跨项目同步时**：需要统一管理散落在不同 GitHub 仓库中的任务时。
- **多任务并行推进时**：手头同时挂着好几个待处理的 Issue 或 PR，并且这些任务的状态经常发生更新时。
- **日常开发执行受阻时**：
 - 今天的任务未按时完成，跨日历切换时。
 - 正在做的任务被阻塞，或发现原有任务的硬截止时间改变时。
 - 上午提交的 PR 在下午收到了修改要求（`changes requested`）或新的未解决意见时。

现有解决方案有什么不足？

- **手动记录与记忆管理的缺陷**：
 - 手动复制 GitHub Issue 耗时耗力，容易遗漏，且状态无法及时联动更新。
 - 纯靠大脑记忆优先级撑不了几天，信息一乱就容易遗忘任务。
 - 第二天手动把未完成任务逐项移动过去，每天重复整理非常繁琐。
- **原生与传统工具体验不佳**：
 - 按 GitHub 默认的创建时间倒序查看，无法体现任务的真实紧急程度（最新 ≠ 最急）。
 - 未完成任务继续留在“今天”，会导致今日列表失真，分不清到底哪些是真正要做的。
 - 简单粗暴地把未完成任务推到“明天”，可能会导致明日任务量直接超过可用时间容量。
 - PR 被退回后，自己手动插回今日列表容易忘记，也可能挤占了更紧急的任务。

5. 产品定位

这是一款专门面向 XEngineer 训练营成员的 GitHub 个人项目管理工具。本工具坚持“GitHub-only”的数据边界，通过单向同步与个人相关的 Issue、PR 和 Review 请求形成个人任务池。产品依托透明的优先级排序规则展示今日待办，并在任务状态变更或发生延期时，自动重排可执行计划。

6. 核心功能列表

任务导入

功能说明：核心围绕目标任务的接入：通过 GitHub 授权绑定后选择单一仓库，将“分配给我且 Open”的 Issue 单向拉取为本地待办；在导入过程中，利用 AI 自动解析 Issue 并为缺少时间信息的任务推荐预计工时，完成任务源数据的闭环导入与初始化。

任务排序

功能说明：核心围绕任务的展示与调度优先级。以**今日计划页**为主阵地，用户可设定 P0-P2 优先级；系统基于“**紧急兜底（≤24h置顶）→优先级→DDL→预估用时**”的四层透明规则自动排序，并辅有无 DDL 任务的**老化上浮机制**。支持展开**任务详情抽屉**查看透明的**排序理由**，同时允许用户在规则框架内进行**手动拖拽排序**微调，实现机器编排与人工干预的完美结合。

任务重排

功能说明：结合**每日容量**、**任务状态（PR联动/阻塞/待估算）**以及**未完成任务**自动生成与更新今日计划。系统支持事件触发的**自动无感重排**，在硬截止超额时触发**冲突处理**，并允许同级任务的**手动微调**，实现任务从规划池到完成核算的全生命周期调度。

7. 功能详细说明

GitHub集成与数据初始化

功能名称：用户认证与 GitHub 账号绑定

功能描述：

提供基础的用户注册、登录功能，并支持直接使用 GitHub 账号进行第三方授权登录（OAuth），以便系统获取用户的仓库和 Issue 数据。

使用场景：

- 用户第一次打开产品，需要创建账号并绑定 GitHub。
- 用户日常使用时的登录。
- GitHub 授权过期，需要重新授权时。

用户流程：

- 用户打开产品，进入欢迎/登录页面。
- 注册/登录：** 选择“账号密码登录/注册”或直接点击“通过 GitHub 登录”。
- 授权：** 如果选择 GitHub 登录，跳转至 GitHub 授权页面，用户同意授予访问 Repository 和 Issue 的基础权限。
- 完成：** 授权成功后返回产品，登录成功并进入下一步（仓库选择）。

规则说明：

- 若后续有其他登录方式，用户通过邮箱注册，登录后必须提示并强制/引导完成 GitHub 账号绑定，否则无法使用核心同步功能。
- 权限获取遵循最小可用原则（仅获取仓库、Issue、PR 等读取权限，不做复杂权限管理）。
- 一个本地账号只能绑定一个 GitHub 账号。

异常情况：

- 网络异常：** 提示“网络连接失败，请检查网络后重试”。
- 授权失败/拒绝：** 提示“获取 GitHub 授权失败，请同意授权以使用后续功能”。
- Token 过期：** 使用过程中如果 GitHub Token 失效，强制登出或弹窗提示“GitHub 授权已过期，请重新登录”。

功能名称：单一仓库选择与单向自动同步

功能描述：

用户登录成功后，系统拉取该 GitHub 账号下的仓库列表。用户选择其中一个仓库后，系统将该仓库中分配给该用户（Assigned to me）的开放状态（Open）的 Issue 单向同步到本地时间管理产品中。

使用场景：

- 首次登录完成授权后，初始化项目数据。
- 用户完成了一个阶段的项目，想要切换到另一个仓库进行任务同步时（如支持解绑/重绑）。

用户流程：

- 页面展示当前绑定的 GitHub 账号下的所有可见 Repository 列表。
- 用户浏览或搜索，选中单个目标仓库。
- 点击“确认/开始同步”按钮。
- 系统展示加载状态，后台开始拉取符合条件的历史 Issue 并将其转化为本地任务（每个任务关联唯一的 `taskSourceId`）。

规则说明：

- 不做全量同步：仅拉取当前处于“开放（Open）”状态且“分配给我（Assigned to me）”的 Issue，已关闭的 Issue 忽略。
- 单向同步：数据方向仅为 GitHub -> 时间管理产品。
- 单一仓库：目前仅支持同时选中并同步一个仓库的数据，不支持多选。
- 不做复杂过滤器：不提供基于标签、里程碑等复杂维度的过滤机制。

异常情况：

- 仓库列表为空：提示“未检测到您有任何可用的 GitHub 仓库”。
- 该仓库下无符合条件的 Issue：同步完成后提示“该仓库下暂无分配给您的开放任务”。
- API 限流：如果触发 GitHub API 请求频率限制，提示“同步请求过于频繁，请稍后再试”。

功能名称：任务初始化与 AI 时间推荐

功能描述：

在 Issue 同步到本地转化为任务的过程中，系统自动提取标题、描述等信息。针对没有明确时间描述的 Issue，调用 AI 能力预测并推荐该任务的“预计开始/结束时间”，最终生成包含时间、状态、优先级的待办列表。

使用场景：

- 仓库同步过程中的自动化数据处理阶段。
- 后续自动检测到新增 Issue 被分配给用户时。

用户流程：

- 系统在后台解析抓取到的 Issue 文本。
- 系统判断 Issue 中是否包含明确的时间点。
- 若无明确时间，系统自动将 Issue 标题/内容发送给 AI，获取推荐的预计耗时和时间节点。
- 任务列表初始化完成，呈现在用户面前，用户可以直观看到带有时间戳（含 AI 推荐标签）的待办项。
- 如果用户认为 AI 推荐不合理，可以直接在列表上点击时间进行手动修改。

规则说明：

- AI 推荐机制：仅针对没有明确时间相关描述的 Issue 生效，推荐结果仅供参考，不强制锁定。
- 不支持历史推断：AI 仅根据当前 Issue 内容进行分析，不通过分析历史已关闭的任务来预测当下耗时。
- 不可手动新增无源任务：任务列表必须基于 Issue 生成，不允许用户在时间管理软件中凭空创建与 Issue 无关的条目。

异常情况：

- **AI 服务超时或调用失败：** 自动降级处理，将任务的预计时间置空，并在界面上提示用户“自动评估时间失败，请手动设置”。
- **Issue 描述过少导致 AI 无法判断：** AI 返回默认保底时间（例如：默认耗时 1 天），并允许用户后续手动调整。

任务优先级排序与展示

功能 01：优先级设置

功能描述： 用户可以给每个任务手动设定 P0 / P1 / P2 优先级，用于参与排序。

使用场景： 用户导入任务后，告诉系统「这件事有多重要」，是排序第一层判断的依据。

用户流程：

1. 用户在任何包含任务卡片的地方（今日计划、任务池、规划池等）找到一个任务。
2. 点击任务上的优先级区域（或右键/长按）展开优先级选择器。
3. 选择 P0（卡住或核心功能）、P1（重要但能商量）、P2（锦上添花），或取消优先级（Unset）。
4. 选择后，优先级立刻生效：卡片上方显示对应的优先级标签。
5. 回到排序列表时，该任务按新优先级重新参与排序。

规则说明：

- P0 = 卡住项或核心功能，不做无法推进下一步；P1 = 重要但时间可商量；P2 = 锦上添花、不紧急。
- 未设置优先级的任务，排序时归入 P2 组末尾。
- 优先级是用户个人的时间管理数据，不写回 GitHub。不会修改 GitHub 上的 Label 或 Issue 属性。
- 优先级可以随时更改，更改后系统无需手动重新排序，排序会自动基于最新优先级重新计算。
- 批量设置：用户可在任务池页勾选多个任务后批量设置优先级。

异常情况：

- **修改优先级后页面未自动刷新：** 等待排序完成（通常 < 500ms），超时后显示提示「排序更新中，请稍候」，不让用户在未排序状态下操作。
- **离线修改优先级：** 修改本地生效，待网络恢复后同步至服务端（如有）。本地优先级始终可用。

功能 02：排序与展示

功能描述： 用户点击排序按钮后，系统按「紧急兜底 → 优先级 → DDL → 预估用时」四层透明规则排序，展示结果。

使用场景： 用户打开「今日计划」页，想知道今天先做什么事。

用户流程：

1. 用户在「今日计划」页点击「排序」按钮。
2. 系统执行四层排序算法，页面任务列表重新排列。
3. 排序完成后，任务按优先级分组展示，组间有分隔线或视觉区分。
4. 每组内部按 DDL 近→远排列（DDL 更近的在上）。
5. 同 DDL 的内部按预估用时短→长排列。
6. $DDL \leq 24h$ 的任务不参与分组，直接置顶到列表最上方，带「紧急」标记。
7. 每个卡片展示：标题、优先级标签（P0/P1/P2）、DDL 或  标记、预估耗时、已耗时。

8. 位置发生变化的任务旁边出现  标记，让用户一眼看到「上次排序之后哪些事挪了位置」。

排序算法（透明链）：

第一步：紧急置顶

DDL ≤ 24h 的任务直接置顶，按 DDL 近→远排列

第二步：按优先级分组

P0 组（卡住或核心功能）
P1 组（重要但能商量）
P2 组（锦上添花）
未设置优先级 → 归入 P2 组末尾

第三步：组内按 DDL 排

有 DDL → 按截止日期近→远
无 DDL → 按老化因子上浮（见功能 07）

第四步：同 DDL 内按预估用时排

有预估用时 → 用时短→长
无预估用时 → 排该档最后

规则说明：

- Issue、PR、Review Request 全部混在同一列表里，共用一套排序规则，不单独分组。
- 排序只影响列表展示，不会修改 GitHub 数据，不会自动改 DDL，不会改计划日期。
- 同条件下顺序稳定——每次排序结果一致，不随机跳变。
- 手动拖拽过的任务在排序时保持相对位置（见功能 06）。
- 已完成的任务自动沉到底部的「已完成」分区，不参与排序。

异常情况：

- **排序时网络中断**：排序逻辑运行在本地（优先级、DDL、预估用时均为本地数据），无需网络也能排序。离线排序结果可正常展示。
- **数据为空（无任务）**：显示空状态文案「还没有任务，先去导入仓库吧」，并提供跳转到仓库选择的入口。
- **排序算法执行超时**：极少发生（数据量通常 < 500 条）。若超时则显示提示「排序异常，请重试」并保持上一次的排序结果。
- **排序结果计算量过大（200+ 任务）**：分页懒加载，不一次性渲染所有卡片。

功能 03：手动拖拽排序

功能描述：用户可以通过拖拽任务卡片，手动调整它们在列表中的顺序。

使用场景：用户觉得系统的排序结果不完全符合自己的直觉，或者有临时安排需要在特定时间做某个任务。

用户流程：

1. 在「今日计划」页，用户长按或按住拖拽手柄，将任务卡片拖到目标位置。
2. 拖拽过程中，列表实时展开空位，显示释放后的预期位置。
3. 释放后，该任务固定在新位置。
4. 用户下次点击「排序」按钮时，未拖拽过的任务重新参与系统排序，拖拽过的任务保持用户设定的相对位置。
5. 新导入的任务或数据变更的任务，按排序规则自动插入到合适位置，不冲乱已拖好的布局。

规则说明：

- 拖拽只记录相对位置，不绑定到绝对索引。例如任务 A 被拖到任务 B 上方，之后 A 和 B 的相对关系保持，但新的任务插入时不会强行塞在中间。
- 拖拽记录的系统内部实现：记录任务的 `manual_order` 字段。排序时，用户拖过的任务之间保持已设定的相对顺序，新任务插在最近的不冲突位置。
- 用户可随时清除所有手动排序记录（在设置或排序菜单中「重置为自动排序」）。
- 同一优先级分组内，拖拽不跨组。用户不能将 P2 的任务拖到 P0 组中。

异常情况：

- **拖拽后页面刷新丢失顺序**：拖拽记录应在每次拖拽操作完成后即时持久化（本地存储），不允许刷新后丢失。
- **跨组拖拽（如把 P2 拖到 P0 组）**：系统不允许，拖到目标位置时自动弹回原位，并提示「只能在同一优先级分组内调整顺序」。
- **网络异常不影响拖拽**：拖拽数据纯本地，不需要网络即可保存。

功能 04：老化因子（无 DDL 任务自动上浮）

功能描述：没有设置截止日期的任务，在任务池中放久了会自动往上浮，防止被用户遗忘。

使用场景：用户有任务没有 DDL（例如「研究 X 技术方案」），但放久了就彻底忘了。系统应该主动把它重新推回用户的视线。

用户流程：

1. 用户给一个任务设定了 DDL → 正常按 DDL 参与排序。
2. 用户没给任务设 DDL → 系统记录该任务自导入或上次处理以来的天数。
3. 排序时，没有 DDL 的任务按「放了多久」逐步上浮：
 - 放了不到 7 天 → 位置不动（排在所属优先级组内但有 DDL 任务的后面）
 - 放了 7-14 天 → 略微上浮
 - 放了 14-21 天 → 明显上浮
 - 放了 21 天以上 → 大幅上浮

规则说明：

- 老化因子只影响「无 DDL 的任务」在有 DDL 任务之间的相对位置。永远不会把无 DDL 的任务排到「紧急置顶」区域。
- 无 DDL 任务在卡片上显示 ∞ 标记（代表无截止日期）。
- 上浮不会跳过同一优先级组内有 DDL 的任务——它只影响无 DDL 任务彼此之间的顺序和它们在组内无 DDL 段的相对提升。
- 用户给任务设置 DDL 后，老化因子对该任务清零，按新 DDL 重新参与排序。
- 用户对无 DDL 任务进行了操作（查看详情、编辑备注等），认为用户已注意到该任务，老化天数重置。

异常情况：

- **老化计算边界**：如导入日期早于 30 天且用户从未操作过，最多上浮到同优先级组内「接近有 DDL 任务的底部」的位置，但不会超越有 DDL 的任务。
- **老化天数计算不准确**：断网状态下老化天数按本地导入时间计算，联网后可以和服务器同步校正。

功能 05：紧急兜底（DDL ≤ 24h 置顶）

功能描述：明天就到截止日期的任务，无论是什么优先级，直接置顶展示。

使用场景：用户的某个任务明天就要交了，但优先级可能只是 P2（锦上添花）。系统必须确保这个任务不会被其他优先级高的任务压到列表深处。

用户流程：

- 1. 用户给一个任务设定了 DDL，该 DDL 距现在 ≤ 24 小时。
- 2. 用户触发排序后，该任务直接从常规排序流程中被提前提取出来：
 - 不参与「按优先级分组」
 - 不参与「按 DDL 排」
 - 不参与「按预估用时排」
- 3. 该任务直接放置在列表最上方，独立于所有优先级分组之上，带有明确「紧急」标记。
- 4. 多个紧急任务之间按 DDL 近→远排列。

规则说明：

- 「紧急」的定义：DDL 距离当前时间 ≤ 24 小时（自然日判断，而非工作日判断）。
- 紧急任务之间仍然按 DDL 紧迫程度排序（精确到小时），最紧急的排最上面。
- 一个紧急任务的优先级标签仍然显示（P0/P1/P2），但不影响它在列表中的位置——「紧急」覆盖优先级。
- 用户如果将紧急任务的 DDL 修改为 > 24 小时，该任务自动退出紧急区域，回到正常排序流程。
- 紧急区域本身不参与手动拖拽的跨区域操作。

异常情况：

- **时区判断**：基于用户本地时区（从系统获取），而非 UTC。DDL 的「明天」是对用户来说的明天。
- **DDL 已过期（小于 0 小时）**：仍视为紧急置顶区域，标记「已过期」而非「紧急」，排序上比未到期紧急任务更靠前。
- **多个任务 DDL 相同**：按预估用时短→长排列。

功能 06：排序理由查看

功能描述：点开任意一个任务，能看到一条一条的排序理由，解释它为什么排在当前这个位置。

使用场景：用户对排序结果有疑问，「为什么 X 排在 Y 前面？」——点开一看就明白了。每一层理由都对应排序算法的一层，用户可以自己对一遍。

用户流程：

- 1. 用户在列表中点击任意任务卡片（或长按选择「查看排序理由」）。
- 2. 系统打开任务详情抽屉，默认或含有一个「排序理由」区域。
- 3. 理由按排序步骤展示，每条独立一行，用自然语言描述：

这件事排在前面是因为：

1. 该任务 DDL 距现在不到 24 小时，紧急置顶

— 或 —

1. 优先级 P0，高于其他 P1 事项

2. 同 P0 组里 DDL 最近（3 天后）

3. 预估只用 1 小时，比同组的其他任务更快搞定

- 4. 每条理由右侧可以点击查看原始数据（点击「P0」看到该任务的优先级字段、点击「3 天后」看到 DDL 的具体日期等）。

规则说明：

- 排序理由根据排序算法的实际结果动态生成，不缓存，不预写。
- 理由展示的数据来源均为本地用户设定数据（优先级、DDL、预估用时），不涉及权重或黑盒计算。
- 如果任务被用户手动拖拽过，排序理由中额外显示一行：「你手动调整了此任务的位置」。
- 如果任务因老化因子上浮，理由中显示：「此任务无 DDL，已放 X 天，因老化因子上浮」。

异常情况：

- **任务数据缺失（无优先级、无 DDL、无预估用时）**：对应层显示「未设定」或「无」，理由中如实反映：如「未设置优先级，归入 P2 组末尾；未设 DDL，按老化因子上浮（已放 10 天）；未填写预估用时，排在同档最后」。
- **理由生成时数据已被修改**：排序理由反映排序时刻的快照数据。如果在查看理由时用户修改了数据，理由不会实时更新——用户可以关闭抽屉再打开以查看最新的理由。

功能 07：任务详情抽屉

功能描述：用户点击任务卡片后打开一个抽屉（侧边栏/浮层），查看任务的完整信息并编辑计划属性。

使用场景：用户需要查看任务的 GitHub 原文、编辑优先级/DDL/预估用时、查看排序理由时使用。

用户流程：

1. 用户在任意页面点击一个任务卡片。
2. 从右侧（或底部）滑出详情抽屉。
3. 抽屉包含以下内容区域：
 - **标题区**：任务标题 + GitHub 编号（#123），点击可在 GitHub 中打开。
 - **状态区**：work_status（进行中 / 待评审 / 已完成）+ plan_bucket（今日/任务池/规划等）。
 - **属性编辑区**：优先级（P0/P1/P2/未设置）、DDL（日期选择器，可选空）、预估用时（数字输入框，小时为单位）、已耗时（显示，不可编辑）。
 - **排序理由区**：见功能 09 的完整内容。
 - **来源区**：来自哪个仓库、GitHub Issue/PR/Review Request 类型、提交人、创建时间。
 - **操作区**：「在 GitHub 中打开」、「移到明日」（弹窗问预估还需要多久）、「标记为已完成」。
4. 用户在抽屉内修改属性后，属性实时保存，关闭抽屉后列表按新属性重排。

规则说明：

- 抽屉为单例模式——同一时间只能打开一个，打开新的自动关闭旧的。
- 抽屉内编辑的属性均属于「个人计划信息」，不会写回 GitHub。
- 修改优先级、DDL、预估用时后，关闭抽屉时自动触发排序更新（若有排序正在展示）。
- 抽屉在移动端为全屏底部弹出（Bottom Sheet），桌面端为右侧抽屉。

异常情况：

- **在 GitHub 中打开时无网络**：提示「无法连接到 GitHub」并提供复制 URL 功能。
- **修改了字段但排序未更新**：手动关闭/重开抽屉后触发重新排序。
- **同时在多端修改（不应发生）**：本地数据以最后一次保存为准。

功能 08：今日计划页（主页面）

功能描述：App 打开后的默认首页，展示排序后的所有待处理任务，是用户最核心的操作界面。

使用场景：用户每次打开 App 的第一眼看到的就是这个页面，决定今天做什么。

用户流程：

1. 用户登录成功后自动进入「今日计划」页。
2. 页面默认展示当前所有 `work_status = on_progress` 的任务，按排序算法排列。
3. 页面顶部有排序按钮（点击触发排序）。

- 4. 列表按「紧急置顶 → P0 → P1 → P2 → 已完成」分区展示。
- 5. 每个任务卡片展示：标题、优先级标签、DDL/∞ 标记、预估耗时、已耗时。
- 6. 用户可以直接在列表上拖拽排序或点击卡片进入详情抽屉。
- 7. 底部 Tab 栏可切换至「任务池」「规划池」「设置」。
- 8. 若存在 $DDL \leq 24h$ 且总预估耗时超过当日容量的任务，弹出容量冲突弹窗。

规则说明：

- 「今日计划」不按日期过滤——所有未完成的（work_status = on_progress）任务都参与。「今日」的含义是「我现在关注的视图」。
- 默认按最近一次排序的结果展示。首次打开或未曾排序时按导入顺序展示（默认排序）。
- 页面支持下拉刷新，刷新时重新执行排序（如数据有变更）。
- 已完成的任务（work_status = done）自动沉到列表底部「已完成」折叠分区。

异常情况：

- **首次使用尚未导入任何仓库：**在今日计划页显示引导卡片「还没有任务，去导入仓库」，点击跳转到仓库选择页。
- **所有任务已完成：**显示空状态页面「今日无事，享受吧 🍷」，并提供跳转到任务池的链接。
- **排序结果巨大（200+ 任务）：**分页加载（每页 50 条），滚动到底部自动加载更多。

自动重排

功能名称：每日容量设置

功能描述：设置用户当天可投入任务的总时间，作为生成今日计划的容量上限。

使用场景：用户首次安排今日任务，或当天可用时间发生变化时。

用户流程：用户进入今日计划，填写或修改每日容量，系统重新计算今日任务。

规则说明：容量必须使用统一时间单位；容量变化会触发自动重排；任务按照 Proposal 02 的排序结果依次装入今日计划。

异常情况：输入为空、非数字或小于零时不保存；容量小于硬截止任务所需时间时，进入“硬截止容量冲突”，不能静默延期。

功能名称：今日计划生成

功能描述：根据任务排序结果、状态、预计投入和每日容量，生成当天可执行的任务列表。

使用场景：用户设置每日容量、进入新的一天，或任务信息发生变化时。

用户流程：用户打开今日计划，系统自动计算并展示当天应执行的任务及剩余容量。

规则说明：只有 `On Progress`、未阻塞且已有预计投入的任务可以进入今日计划；排序必须复用 Proposal 02，不能建立另一套优先级规则。

异常情况：没有符合条件的任务时展示空计划；缺少预计投入的任务进入“待估算”；同步失败时保留最近一次有效计划并提示数据可能不是最新状态。

功能名称：事件触发自动重排

功能描述：在任务或计划条件变化后，自动更新今日计划。

使用场景：新任务进入、PR 状态变化、任务阻塞或解除、优先级或截止变化、容量变化、任务完成或日期切换时。

用户流程：用户无需点击“重新规划”；系统感知变化、重新计算计划并展示结果。

规则说明：自动重排直接生效，不要求逐项确认；重排只能改变 App 内计划位置，不能修改 GitHub 状态、硬截止、预计投入或任务内容。

异常情况：GitHub 事件同步失败时不基于不完整数据重排；保留原计划并提示同步异常，恢复后重新计算。

功能名称：PR 状态联动

功能描述：根据 PR 提交、Review、合并、关闭和 Reopen 状态，更新任务主状态及计划位置。

使用场景：用户提交 PR、收到 Review 意见、完成返工、PR 合并或关闭时。

用户流程：用户在 GitHub 推进 PR，App 自动同步状态并调整任务。

规则说明：

- 提交 PR或完成返工并重新请求 Review：On Progress → Pending Review；
- 收到 changes requested 或明确修改要求：Pending Review → On Progress；
- approved 或无需修改的普通评论：保持 Pending Review；
- PR 合并或关闭：进入 Done；
- 合并与关闭未合并分别显示“已合并”和“已关闭未合并”。

异常情况：无法判断普通评论是否要求修改时，不自动回到 On Progress；状态同步失败时保持原状态并提示同步异常。

功能名称：任务 Reopen 恢复

功能描述：当已关闭但未合并的 PR 被重新打开，或已关闭的 Issue 被重新打开时，任务恢复为可执行状态。

使用场景：任务关闭后重新进入处理范围，或未合并 PR 被 Reopen。

用户流程：用户在 GitHub Reopen 对象，App 自动把任务恢复为 On Progress 并重新参与计划。

规则说明：恢复原有优先级、硬截止、预计投入和手动顺序；恢复后重新参与 Proposal 02 排序和容量安排。

异常情况：原规划数据不存在时，任务进入任务池并标记缺失字段；缺少预计投入时进入“待估算”，不能直接进入今日计划。

功能名称：规划池管理

功能描述：保存仍需处理、但当前没有进入今日计划的任务。

使用场景：今日容量不足、新的高优先任务进入，或任务暂时未被选入今日计划时。

用户流程：系统自动将较低优先任务移入规划池；用户可以打开规划池查看任务和移入原因。

规则说明：移入规划池不代表删除、完成或关闭任务；任务不会自动安排到后天及更远日期；日期切换时重新参与下一日安排。

异常情况：任务移动失败时保留原计划位置并提示更新失败；不得因此删除对应任务或关闭 GitHub Issue。

功能名称：阻塞任务管理

功能描述：将当前无法继续执行的任务移出可执行计划，并在阻塞区域集中展示。

使用场景：用户主动标记任务阻塞，或 GitHub Issue 出现约定的 blocked 标记时。

用户流程：用户标记阻塞，任务离开今日计划；解除阻塞后，任务自动重新参与排序和安排。

规则说明：阻塞任务不占用今日容量；阻塞只是状态原因和计划位置，不新增主状态；解除阻塞会触发自动重排。

异常情况：GitHub 阻塞信息无法同步时保留最近状态并提示；解除阻塞后若缺少预计投入，任务进入“待估算”。

功能名称：待估算任务管理

功能描述：集中展示缺少预计投入、暂时不能进行容量安排的任务。

使用场景：新任务没有预计投入，或返工任务尚未确定需要多少时间时。

用户流程：用户进入待估算区域，补充预计投入；保存后任务自动重新参与今日计划计算。

规则说明：待估算任务可以显示优先级，但不能进入今日计划；系统不能自行虚构预计投入。

异常情况：输入为空、非数字或小于等于零时不保存；保存失败时保留“待估算”状态，不参与容量安排。

功能名称：任务手动排序

功能描述：允许用户调整同等排序条件下的任务先后顺序。

使用场景：自动重排完成后，用户希望根据个人判断调整执行顺序。

用户流程：用户拖动任务到目标位置，系统保存相对顺序，并更新今日计划展示。

规则说明：手动顺序仅在相同硬截止或同等排序条件内保持；更早的硬截止或紧急层级可以覆盖手动顺序，但必须说明原因。

异常情况：拖动到不合法位置时恢复原顺序；保存失败时提示用户并保持最近一次成功顺序。

功能名称：重排变化摘要

功能描述：解释自动重排后哪些任务和计划发生了变化。

使用场景：任何自动重排完成后，用户需要理解新计划与原计划的差异。

用户流程：用户查看变化摘要，了解任务被加入、移出、提前或改变状态的原因。

规则说明：摘要至少包含新增任务、移出任务、顺序变化、状态变化和对应触发原因；每项变化应能定位到具体任务。

异常情况：重排未产生变化时显示“计划无变化”；无法获得完整原因时不能编造解释，应显示“原因数据暂不可用”。

功能名称：硬截止容量冲突处理

功能描述：当必须今日完成的任务超过剩余容量时，明确展示冲突并让用户决定如何处理。

使用场景：硬截止任务进入今日计划，但所需总时间超过当天剩余容量。

用户流程：用户查看冲突任务、所需时间和容量缺口，然后选择调高容量、调整任务范围或预计投入、手动移出任务，或接受可能延期。

规则说明：系统不能静默延期、修改硬截止或隐藏冲突；用户作出选择前持续显示冲突状态。

异常情况：任务缺少预计投入时不能计算准确缺口，应先进入“待估算”；容量数据异常时停止自动安排并提示修正。

功能名称：跨日未完成任务重排

功能描述：日期切换后，自动重新安排昨日未完成任务。

使用场景：一天结束时仍有任务未完成，用户进入下一天的计划。

用户流程：用户无需逐项顺延；系统将未完成任务与规划池任务一起重新排序，并生成新一天计划。

规则说明：不能把所有未完成任务无条件放入下一天；必须结合 Proposal 02 的排序结果、下一日容量、阻塞状态和预计投入重新安排。

异常情况：下一日容量未设置时不能生成最终计划，应提示用户设置容量；硬截止任务超过容量时进入冲突处理。

功能名称：任务完成与容量核算

功能描述：任务完成后更新任务状态，并计算当天已经使用的容量。

使用场景：Issue 关闭、PR 合并或任务不再需要用户行动时。

用户流程：系统同步完成状态，将任务移出可执行计划并更新剩余容量。

异常情况：任务没有预计投入时无法准确核算容量，应标记核算信息缺失；GitHub 完成状态同步失败时保持原状态并提示。

10.1 页面结构总览

页面	页面类型	负责什么	不负责什么
S01 连接 GitHub	首次配置	说明产品只读 GitHub，并让用户完成授权入口。	不展示任务，不展示工作台，不做功能介绍页。
S02 选择工作仓库	首次配置	选择一个当前要管理的 GitHub 仓库。	不做多仓库总览，不做团队项目列表。
S03 同步与时间建议	首次同步	展示同步结果，确认缺失的预计投入等个人规划字段。	不自动生成完整计划，不把建议值当事实。
S04 全部待办	个人任务池	汇总与我有关的 Issue、PR、Review，并展示当前优先顺序。	不等同于今日计划，不做团队看板。
S05 任务详情	任务证据与规划	左侧展示 GitHub 只读事实，右侧展示个人规划字段。	不修改 GitHub，不提交 Review，不关闭任务。
S06 今日计划	今日执行入口	展示今日容量、今日任务、待评审、规划池、待估算入口。	不做完整日历，不做番茄钟，不做长期排程。
S07 自动重排 / 计划已更新	变化回执	展示计划变化：加入、移出、顺序变化，以及原因。	不隐藏变化，不只给一个“已更新”提示。
S08 硬截止容量冲突	决策处理	当今日硬截止任务超过容量时，让用户选择处理方向。	不自动延期，不自动提高容量，不替用户承诺完成。

10.2 信息层级

首版信息分四层：

- 1. **工作上下文**：当前 GitHub 账号、仓库、同步状态。
- 2. **GitHub 事实**：Issue、PR、Review、状态、负责人、Milestone、更新时间、原始链接。
- 3. **个人规划字段**：优先级、硬截止、预计投入、当前安排、手动顺序。
- 4. **计划结果**：今日计划、规划池、待估算、待评审、计划变化回执、容量冲突。

GitHub 事实和个人规划字段必须分开。GitHub 事实只读，个人规划只保存在产品内。页面不能让用户误以为修改了产品里的字段就会改 GitHub。

10.3 导航结构

参考原型有两套导航：

导航	出现位置	作用
顶部 8 步跳转	全局原型窗口	用于评审快速切换原型页面，不是正式产品主导航。
左侧工作区导航	S04-S06	用于真实工作台内切换：今日计划、全部待办、待评审、阻塞任务。

左侧工作区导航只在正式工作台页出现。S01-S03 是首次配置和同步页，S07-S08 是焦点决策页，都不显示工作台导航。

10.4 页面之间的关系

首次进入：
S01 连接 GitHub
-> S02 选择工作仓库
-> S03 同步与时间建议
-> S06 今日计划

日常查看：

S06 今日计划

-> S04 全部待办

-> S05 任务详情

-> GitHub 原始链接

状态变化：

S06 今日计划

-> S07 自动重排 / 计划已更新

-> S08 硬截止容量冲突

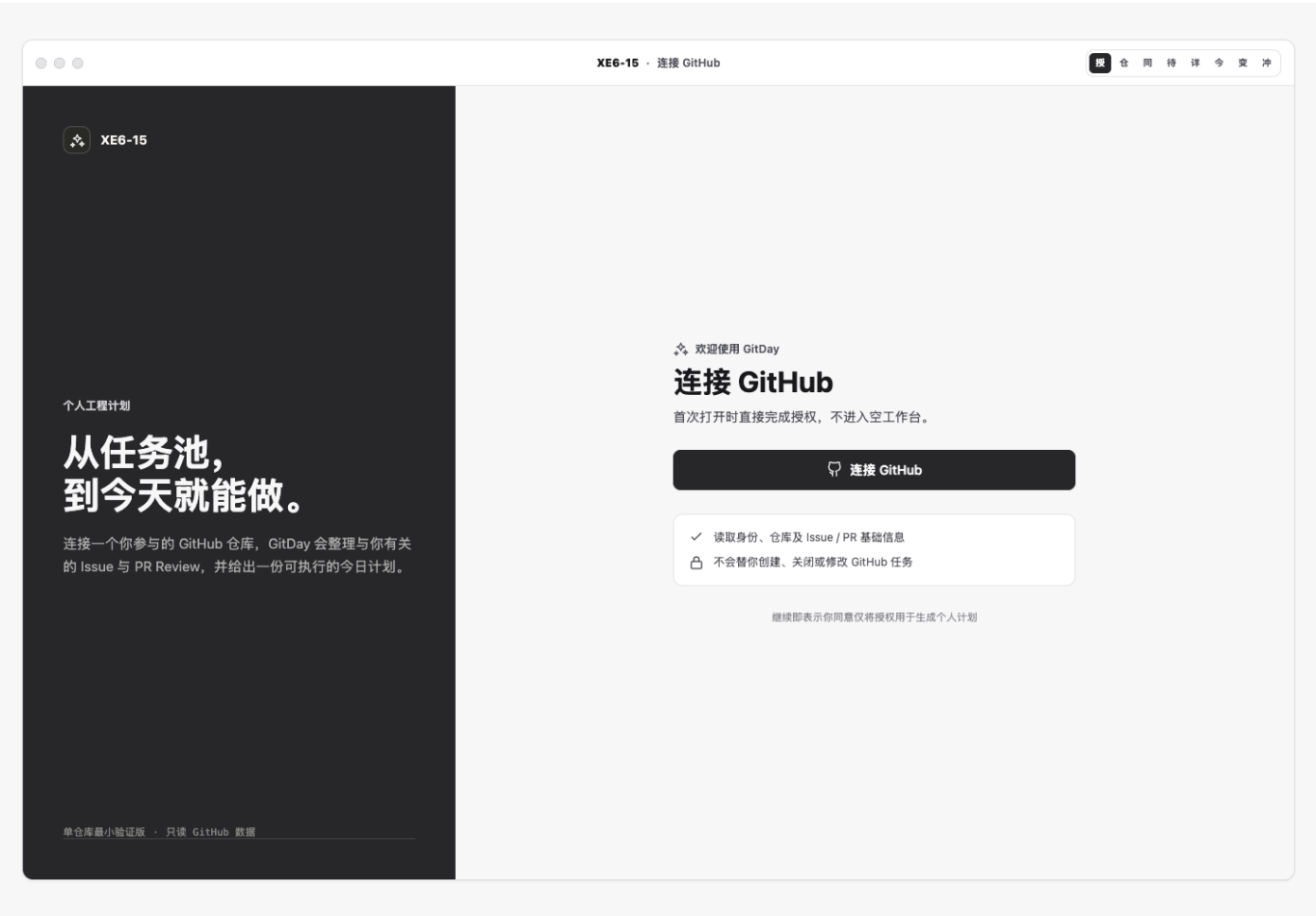
-> S06 今日计划

首页不是传统意义上的营销首页。进入工作台后，用户最重要的首页是 S06 今日计划，因为它直接回答“今天先做什么”。

10. 交互原型/页面说明

S01 连接 GitHub

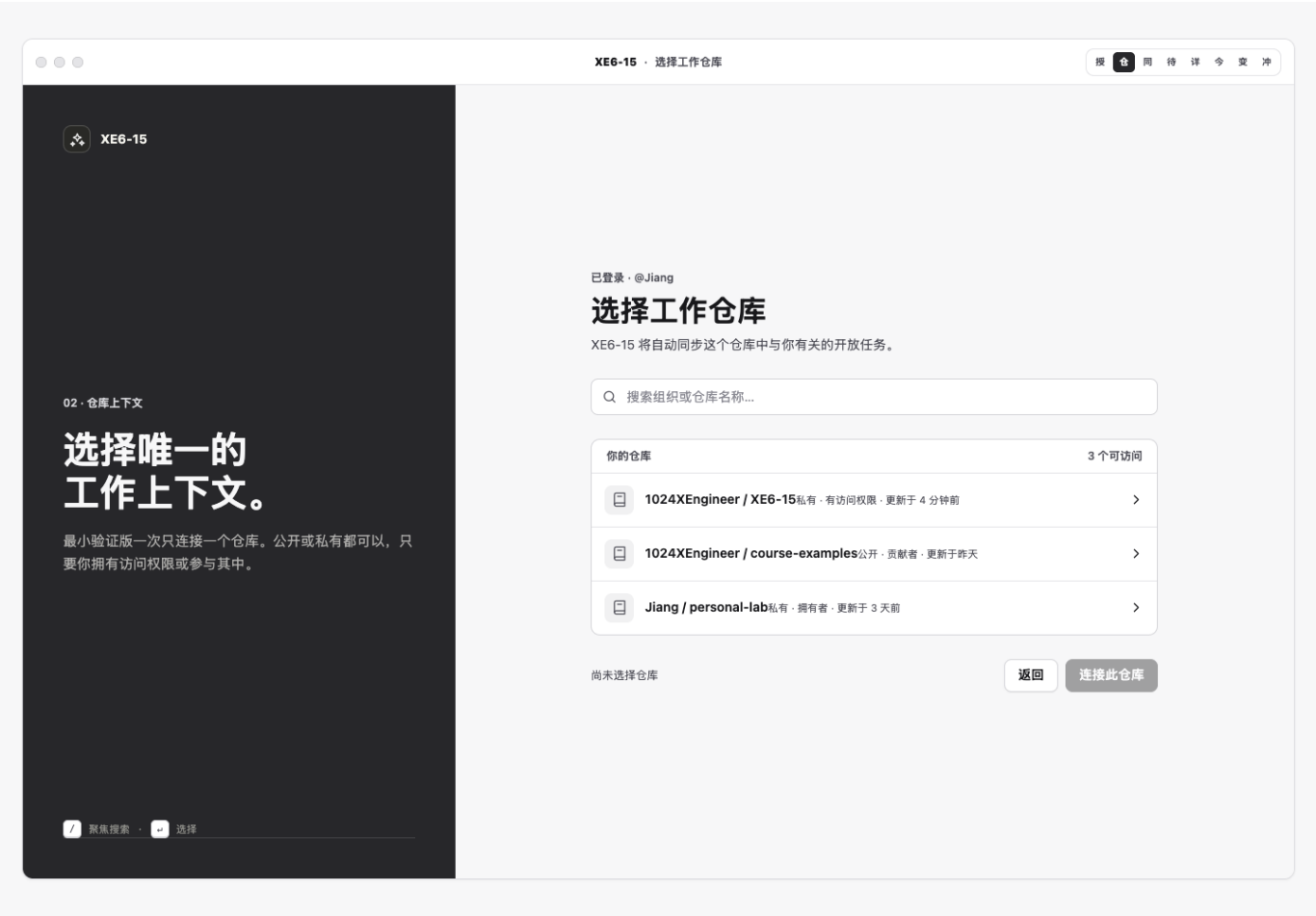
项目	说明
页面目标	让用户明确产品依赖 GitHub，并完成授权入口。
页面内容	品牌 XE6-15、产品说明、连接 GitHub 按钮、只读权限说明、不会修改 GitHub 的承诺。
用户操作	点击 连接 GitHub。
跳转	进入 S02 选择工作仓库。



验收重点：这一页不能变成营销页。它只回答“为什么要连 GitHub、会读什么、不会改什么”。

S02 选择工作仓库

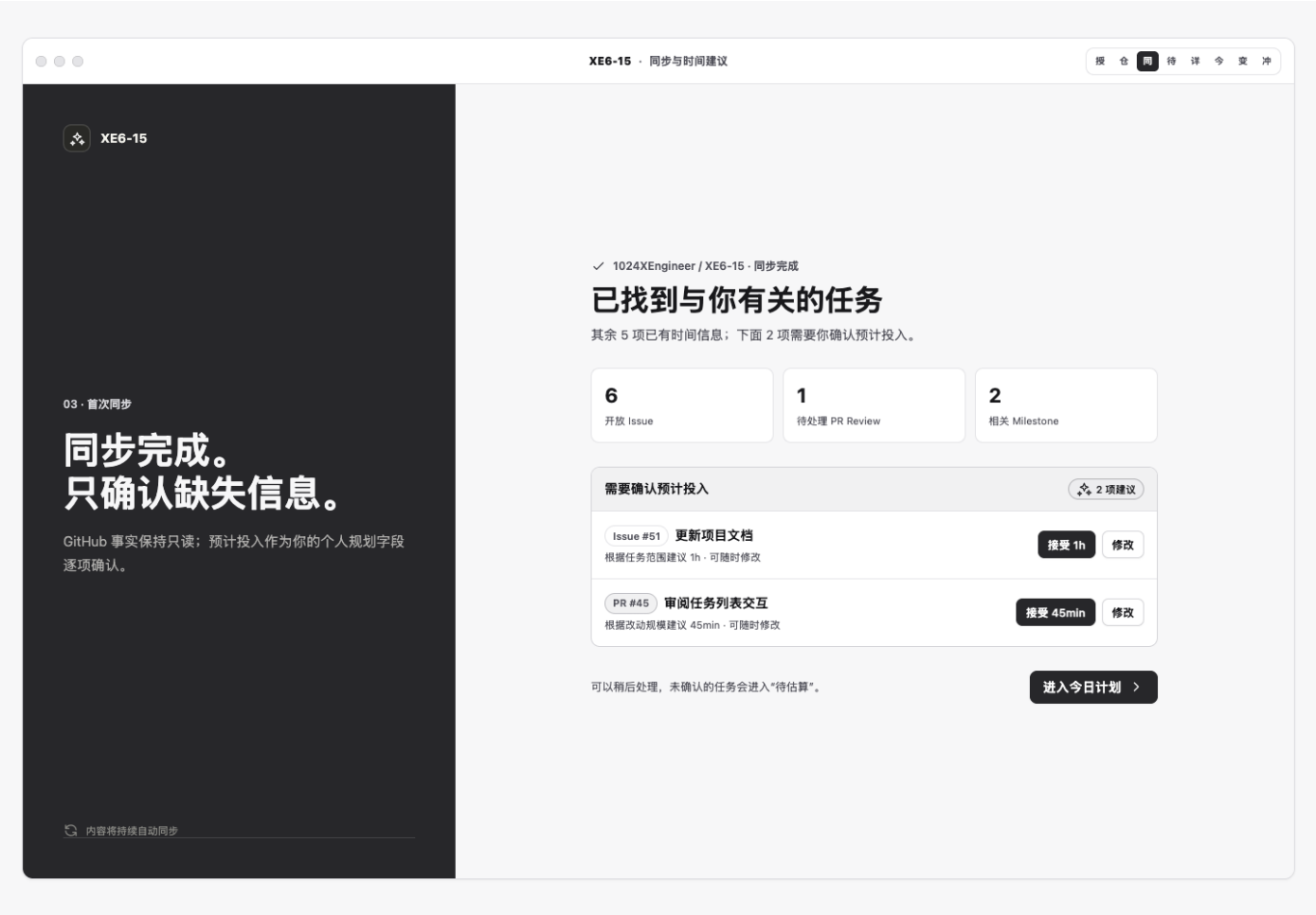
项目	说明
页面目标	选择唯一的当前工作仓库。
页面内容	当前登录用户、仓库搜索框、可访问仓库列表、已选择仓库提示、 连接此仓库 按钮。
用户操作	搜索仓库、选择仓库、返回上一步、确认连接。
跳转	选择仓库后进入 S03 同步与时间建议。



验收重点：可以展示多个候选仓库，但首版只连接一个仓库，不进入多仓库管理。

S03 同步与时间建议

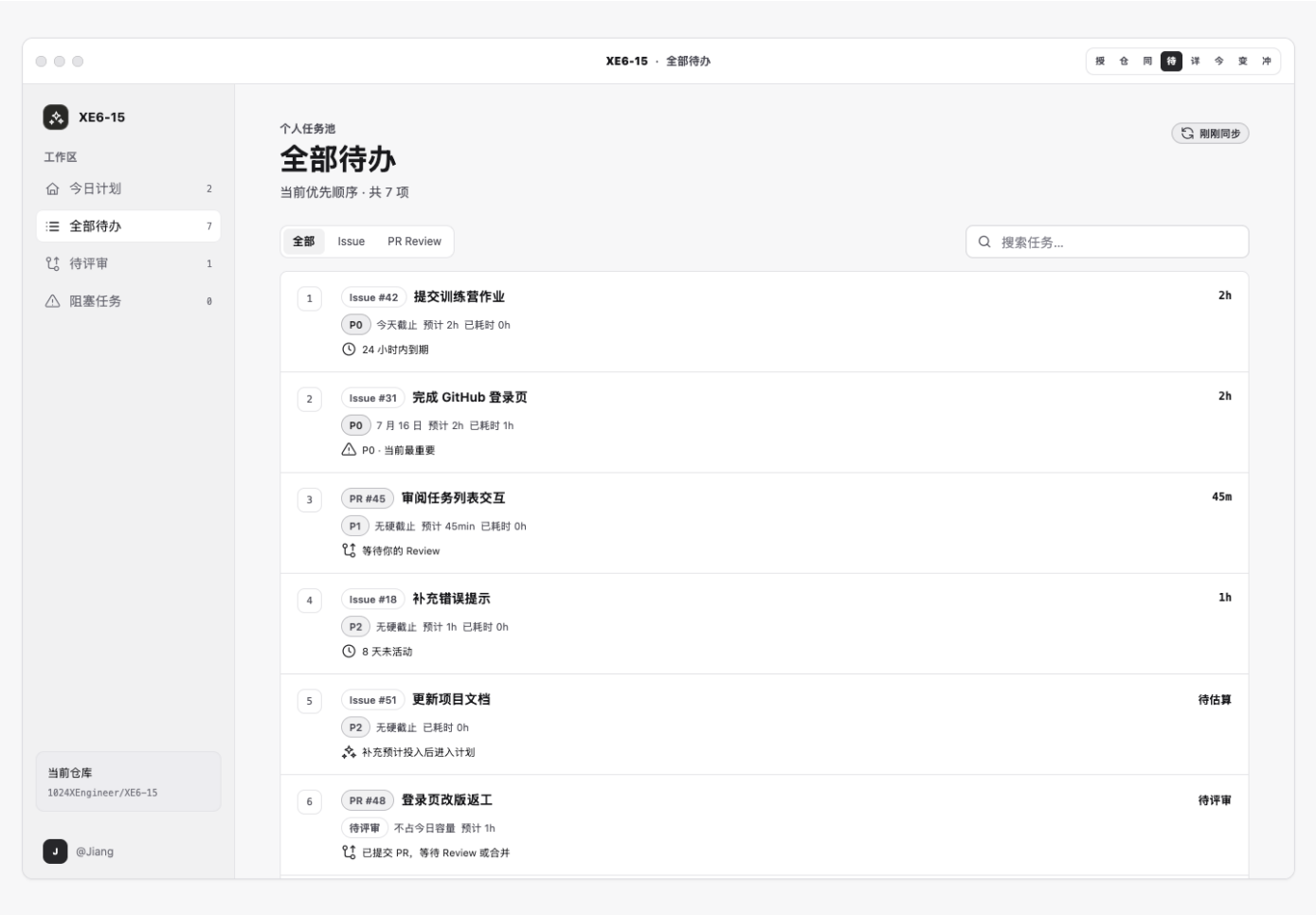
项目	说明
页面目标	展示同步结果，并让用户确认缺失的个人规划字段。
页面内容	同步完成状态、开放 Issue 数、待处理 PR Review 数、相关 Milestone 数、需要确认预计投入的任务。
用户操作	接受预计投入、修改预计投入、保存、跳过后进入今日计划。
跳转	点击 进入今日计划 后进入 S06。



验收重点：预计投入是用户确认字段，不是 GitHub 事实。用户没有确认的任务应进入待估算，不能直接参与今日容量计算。

S04 全部待办

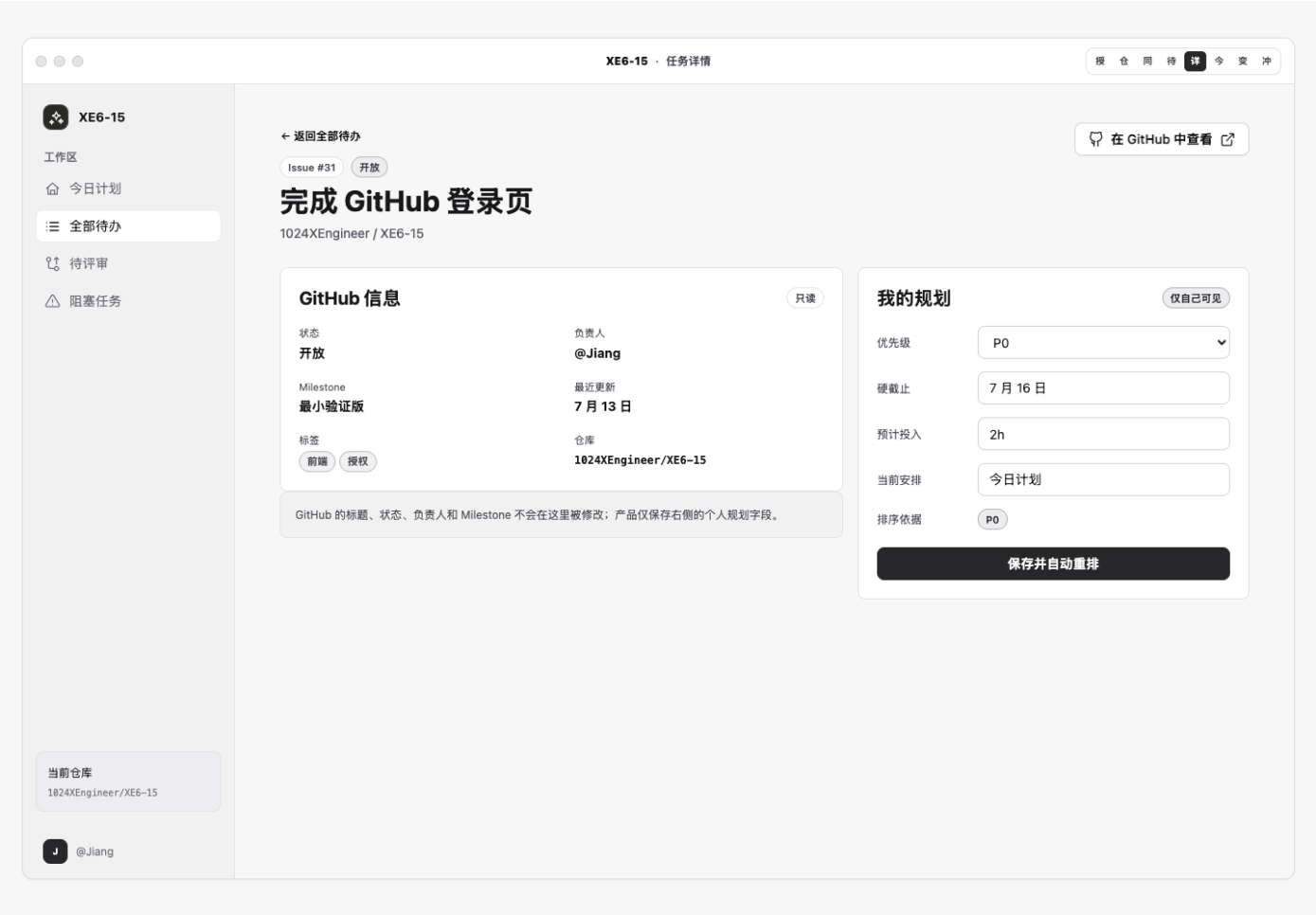
项目	说明
页面目标	展示所有与我有关的个人行动项和当前优先顺序。
页面内容	左侧工作区导航、当前仓库、当前用户、任务筛选、搜索、任务列表、排序原因、预计投入、状态标签。
用户操作	按全部 / Issue / PR Review 筛选，搜索任务，点击任务进入详情。
跳转	点击任务进入 S05；左侧可进入 S06。



验收重点：这里是个人任务池，不是今日计划。任务可以很多，但每项都必须说明来源和为什么与我有关。

S05 任务详情

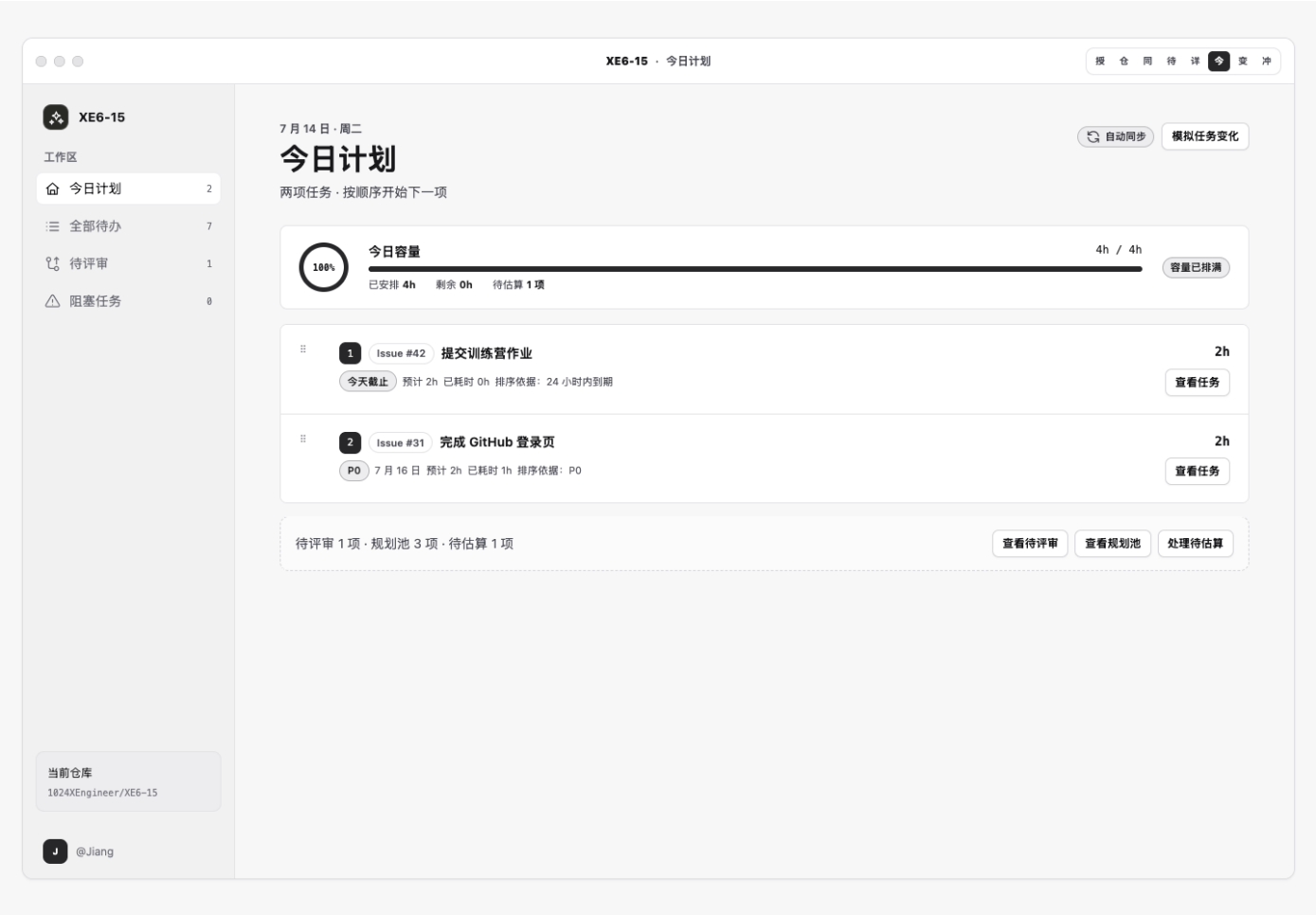
项目	说明
页面目标	把 GitHub 事实和个人规划字段分开展示。
页面内容	Issue 编号、标题、GitHub 链接、状态、负责人、Milestone、最近更新、标签、仓库、个人优先级、硬截止、预计投入、当前安排、排序依据。
用户操作	返回全部待办、打开 GitHub 原始页面、修改个人规划字段、保存并重排。
跳转	返回 S04；保存后可进入 S07 展示计划变化。



验收重点：GitHub 信息只读。个人规划字段可改，但不能暗示会同步回 GitHub。

S06 今日计划

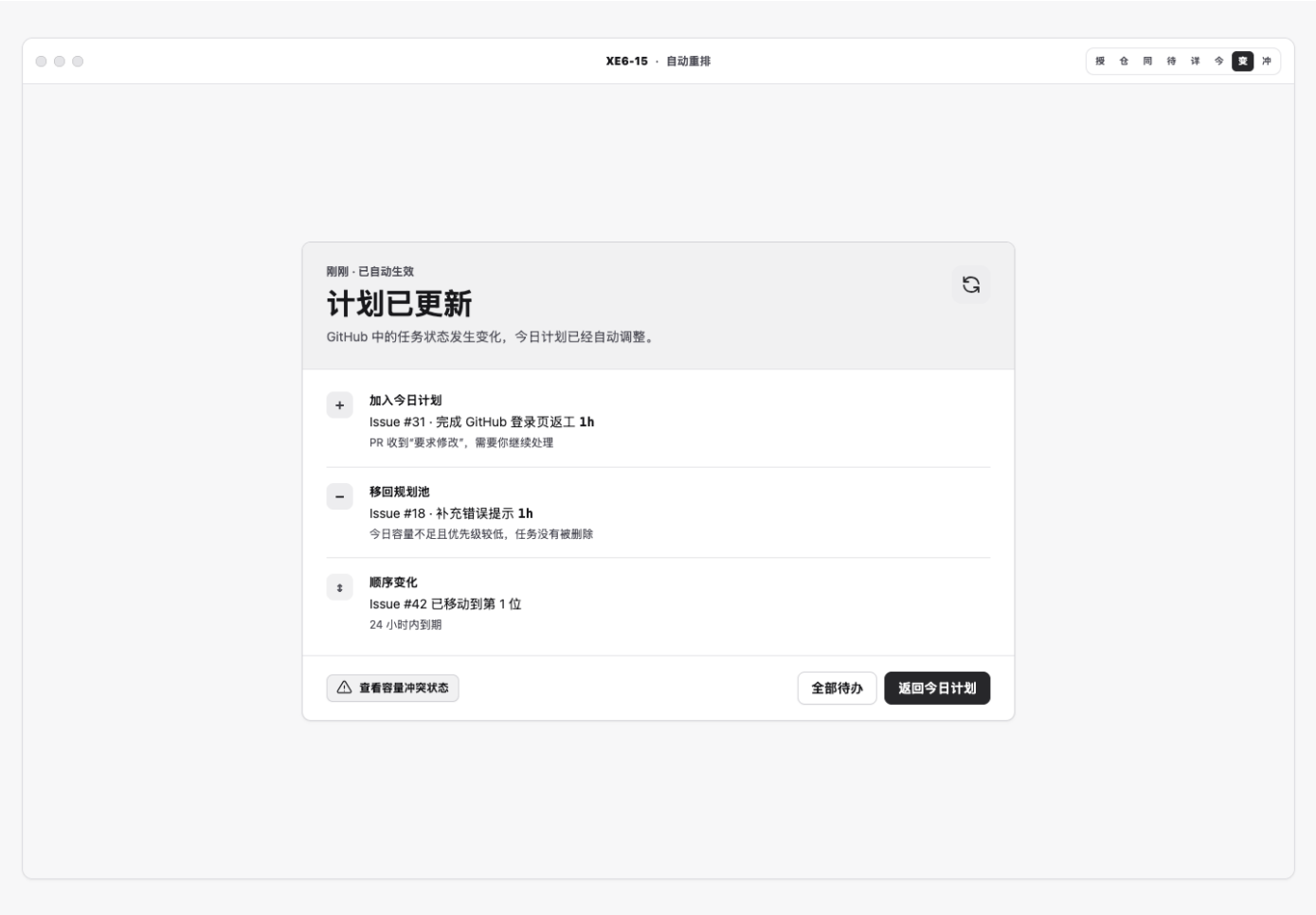
项目	说明
页面目标	告诉用户今天先做什么，以及容量是否够。
页面内容	今日日期、自动同步状态、今日容量、已安排时间、剩余时间、待估算数量、今日任务列表、排序依据、待评审 / 规划池 / 待估算入口。
用户操作	查看任务、拖动今日任务顺序、进入任务详情、查看规划池、处理待估算、模拟任务变化。
跳转	点击任务进入 S05；模拟变化进入 S07；查看规划池进入 S04；处理待估算进入 S03。



验收重点：S06 是工作台的核心首页。它不需要展示所有功能，只要把“今天先做什么”讲清楚。

S07 自动重排 / 计划已更新

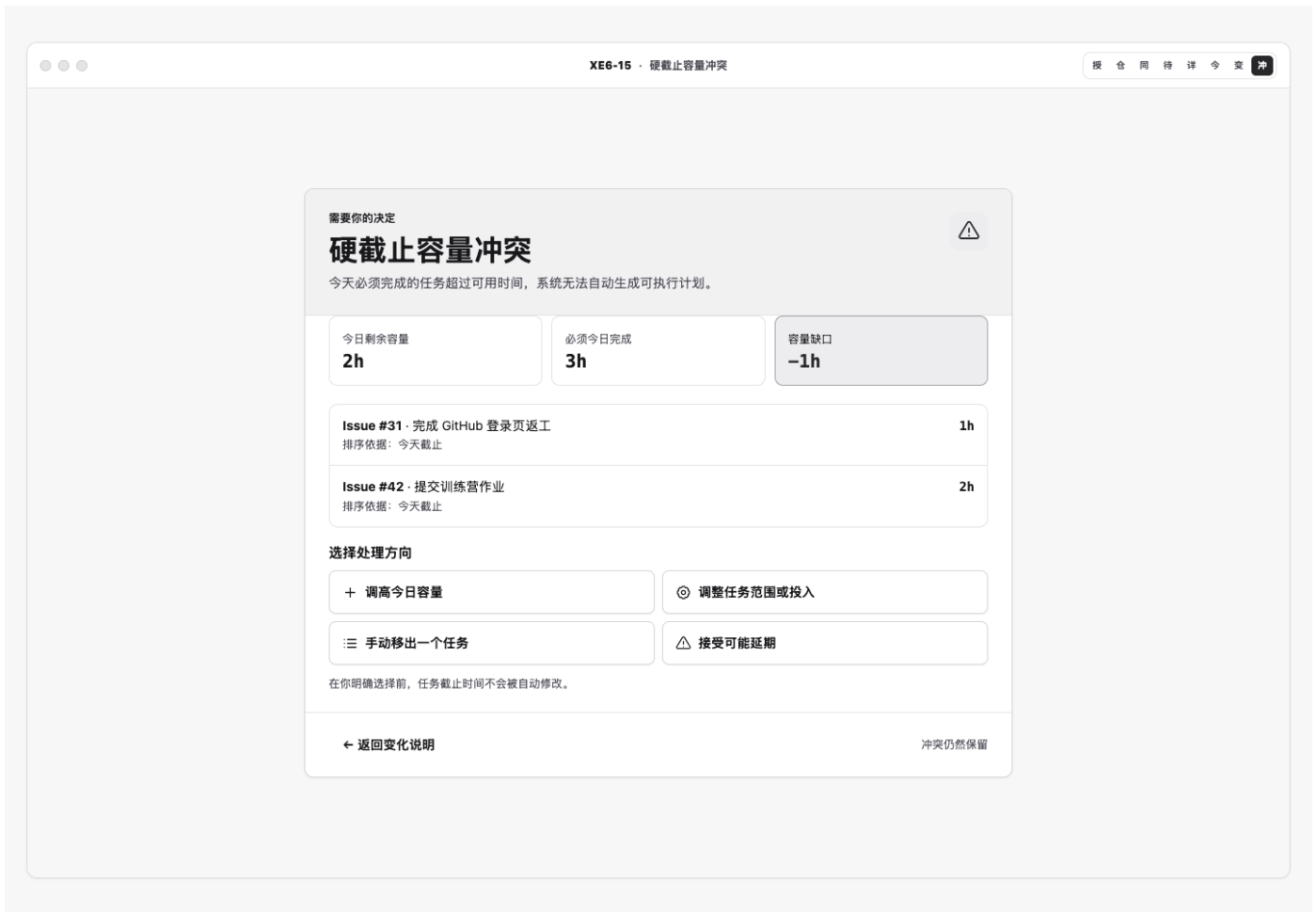
项目	说明
页面目标	在计划变化后给用户一张清楚的变化回执。
页面内容	计划已更新 标题、加入今日计划、移回规划池、顺序变化、每项变化原因、返回今日计划入口、查看容量冲突入口。
用户操作	查看变化原因，返回今日计划，查看容量冲突状态，进入全部待办。
跳转	返回 S06；进入 S08；进入 S04。



验收重点：自动重排可以演示为“已生效”，但不能没有回执。用户必须知道哪个任务变了、为什么变。

S08 硬截止容量冲突

项目	说明
页面目标	当必须今日完成的任务超过容量时，让用户明确做取舍。
页面内容	今日剩余容量、必须今日完成的任务总量、容量缺口、冲突任务列表、四个处理方向。
用户操作	选择调高今日容量、调整任务范围或投入、手动移出任务、接受可能延期；确认后重新计算。
跳转	确认后回到 S06；调整任务范围时进入 S05；可返回 S07。



12. 验收标准

- 用户可以成功进行登录操作。
- 用户可以查看仓库列表。
- 能够初始化任务列表，并且所有待办项的基础数据（如：时间、状态）均初始化完毕。
- 用户可以在不手动重新规划的情况下，分别完成一次当日重排和一次跨日重排体验。
- 每项自动变化都有用户能理解的原因。
- 自动重排不会修改 GitHub 状态、硬截止、预计投入或任务内容。
- 从今日计划移出的任务仍可在规划池找到。
- 用户不需要记录实际投入，也不需要逐项确认重排结果。
- 用户可以完整看到 On Progress → Pending Review → On Progress → Pending Review → Done 的
- PR 状态流转，并在 Reopen 后看到任务重新参与排序。
- 设好 P0/P1/P2 后，列表按优先级正确分组，确保 P0 在上、P2 在下。
- （即将）逾期的排在未逾期的前面，DDL 更近的更靠前。
- 点开任意事项，都能看到它为什么排在当前位置。
- 没设 DDL 的任务正常显示，带 ∞ 标记。
- 没填预估耗时的显示“待估算”，系统不替用户瞎编数字。
- 同样条件下顺序稳定，每次刷新不随机乱跳。
- 排序只影响列表展示，不动 GitHub 数据、不自动改 DDL、不改计划日期。

- 排序结果可以被 Proposal 03 读取并用于重排。
- Demo 能跑通：导入四种时间状态的任务 -> 点排序得到顺序 -> 查看每项的排序理由。
- 别人要求 Review 的 PR 跟自己的 Issue、PR 在同一个列表里混排。